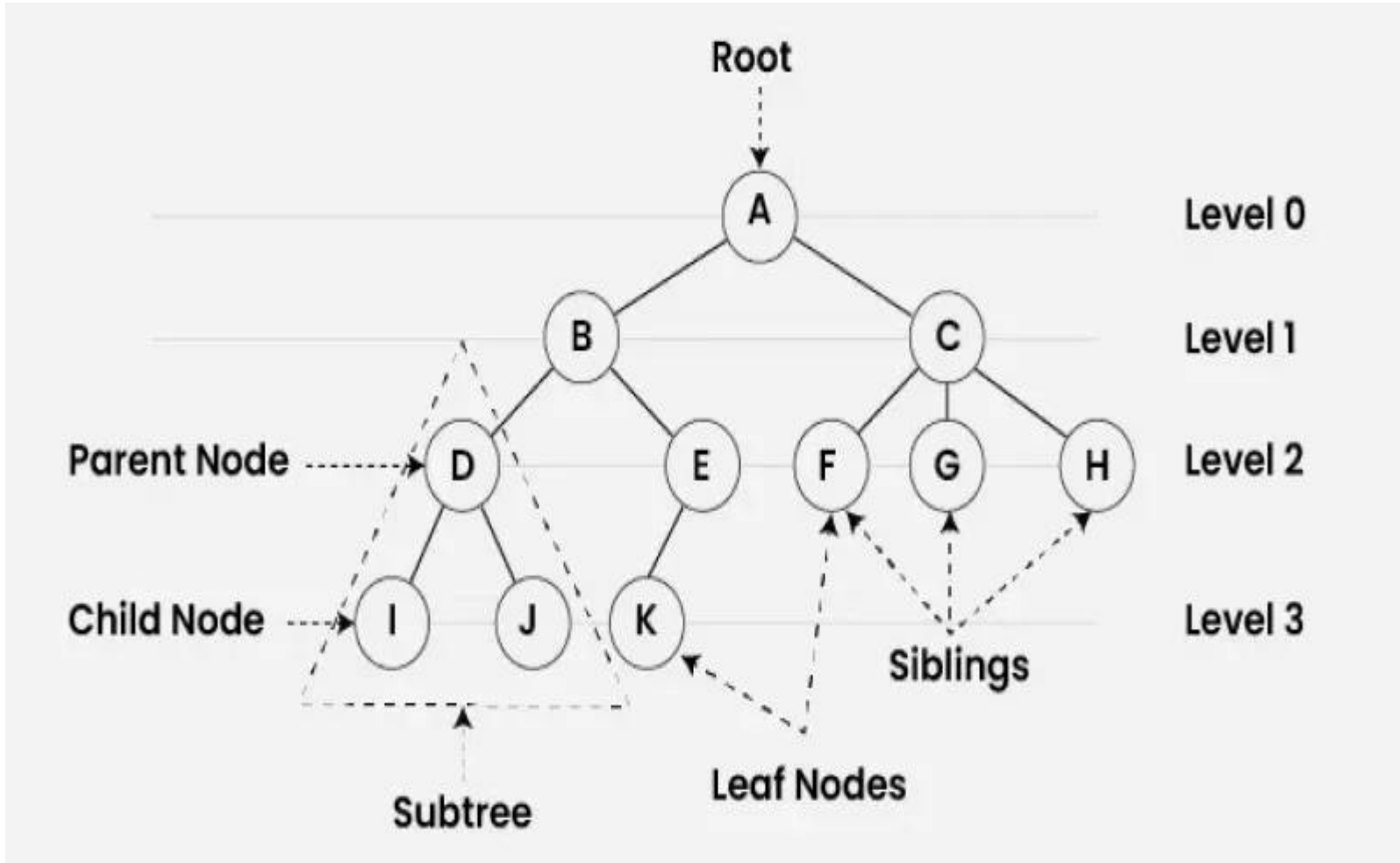
Trees and tree traversals

Tree Data Structure

Tree Data Structure is a non-linear data structure in which a collection of elements known as nodes are connected to each other via edges such that there exists exactly one path between any two nodes.

OR

A tree is a hierarchical data structure that consists of nodes connected by edges. It is used to represent relationships between elements, where each node holds data and is connected to other nodes with edges.


Root
A
B
C
D
E
F
G
H
I
J
K
Level 0
Level 1
Level 2
Level 3
Parent Node
Child Node
Siblings
Subtree
Leaf Nodes

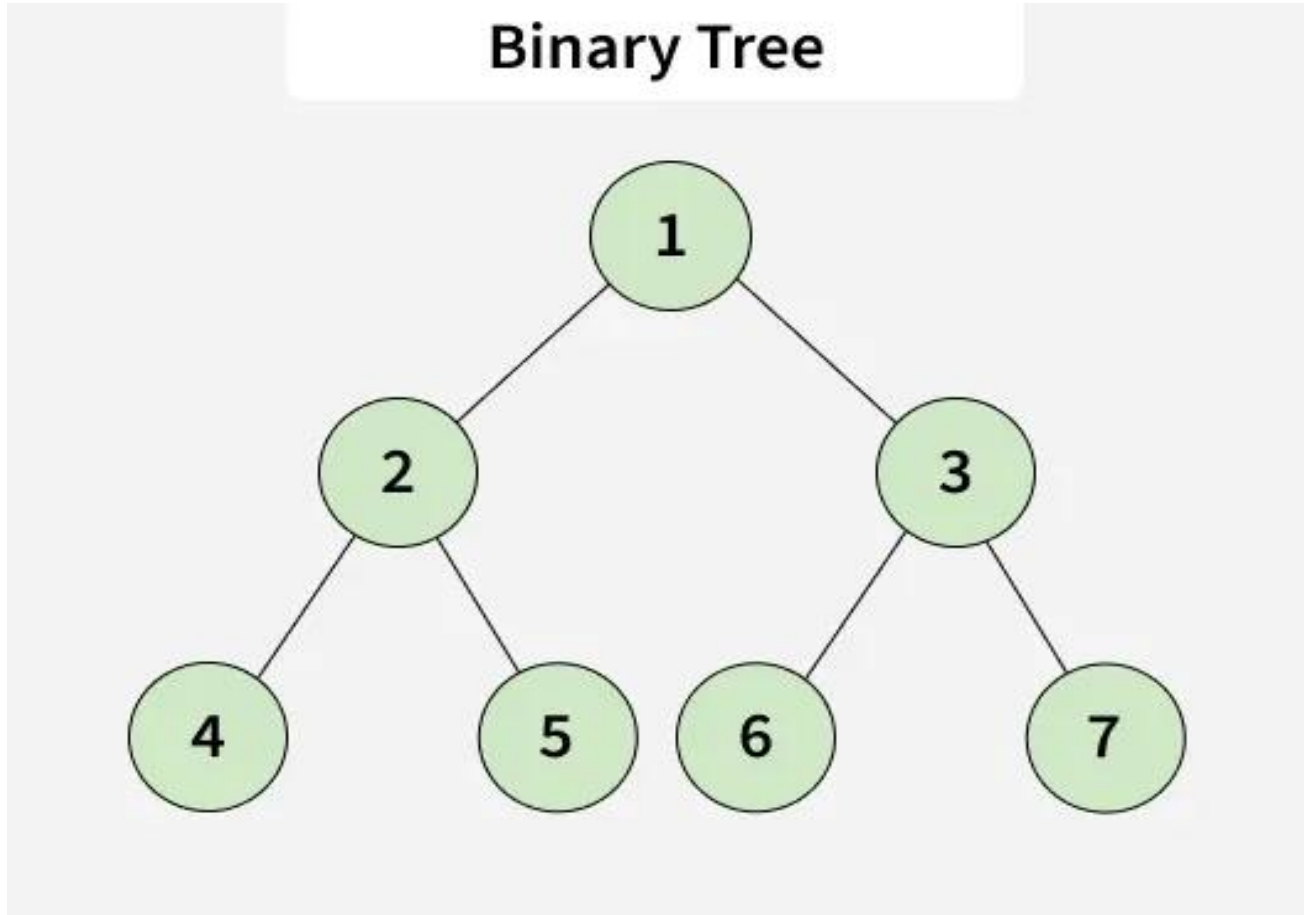
Types of Tree

- **Binary Tree :** Every node has at most two children
- **Ternary Tree :** Every node has at most three children

1. Binary Tree

A binary tree is a tree data structure where each node has at most two children. These two children are usually referred to as the left child and right child.

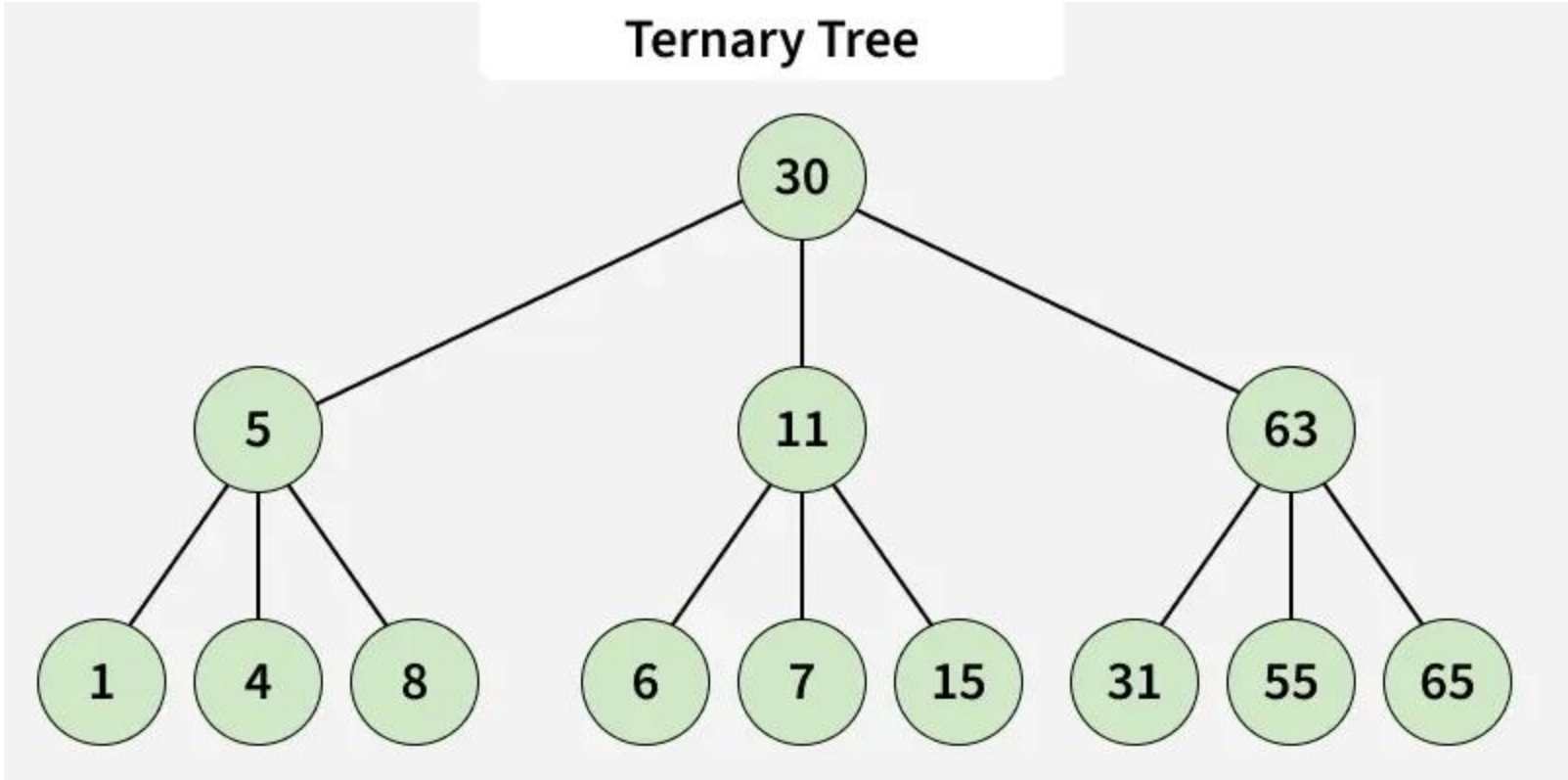
Example: Consider the tree below. Since each node of this tree has at most 2 children, it can be said that this tree is a Binary Tree.


Binary Tree
1
2
3
4
5
6
7

2. Ternary Tree

A Ternary Tree is a tree data structure in which each node has at most three children. These three children are usually referred to as the left child, mid child and right child.

Example: Consider the tree below. Since each node of this tree has only 3 children, it can be said that this tree is a Ternary Tree.


Ternary Tree
30
5
11
63
1
4
8
6
7
15
31
55
65

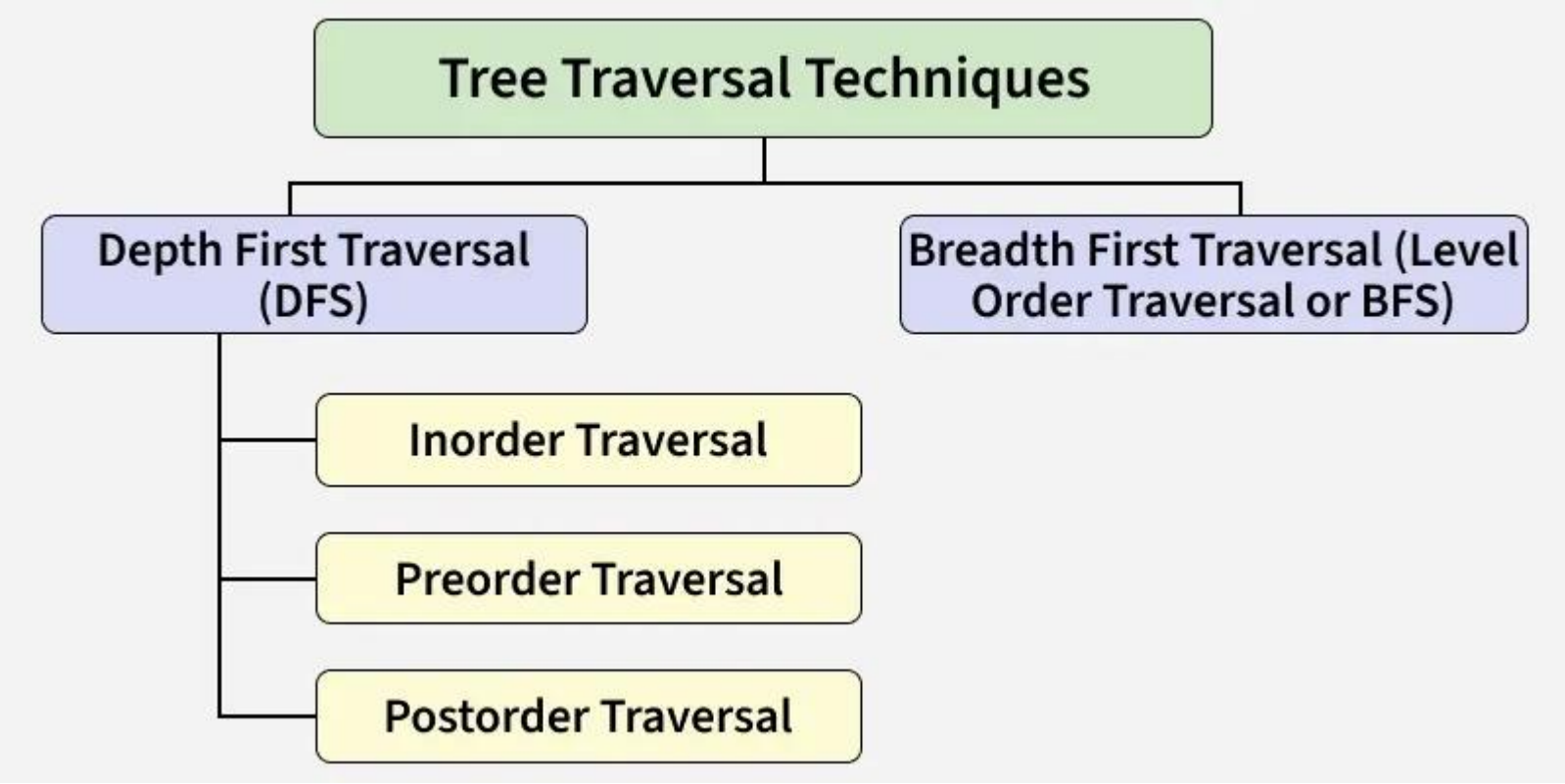
Tree Traversal Techniques

Tree traversal refers to the process of visiting or accessing each node of a tree exactly once in a specific order. Unlike linear data structures such as arrays, linked lists, or queues (which have only one logical way of traversal), trees offer multiple ways to traverse their nodes.

Tree traversals are broadly classified into two categories:

Breadth-First Traversal (BFT)

- Explores nodes level by level from top to bottom.
- Type: Level Order Traversal.
- The level order traversal of the above tree is 1, 2, 3, 4, 5, 6, and 7.

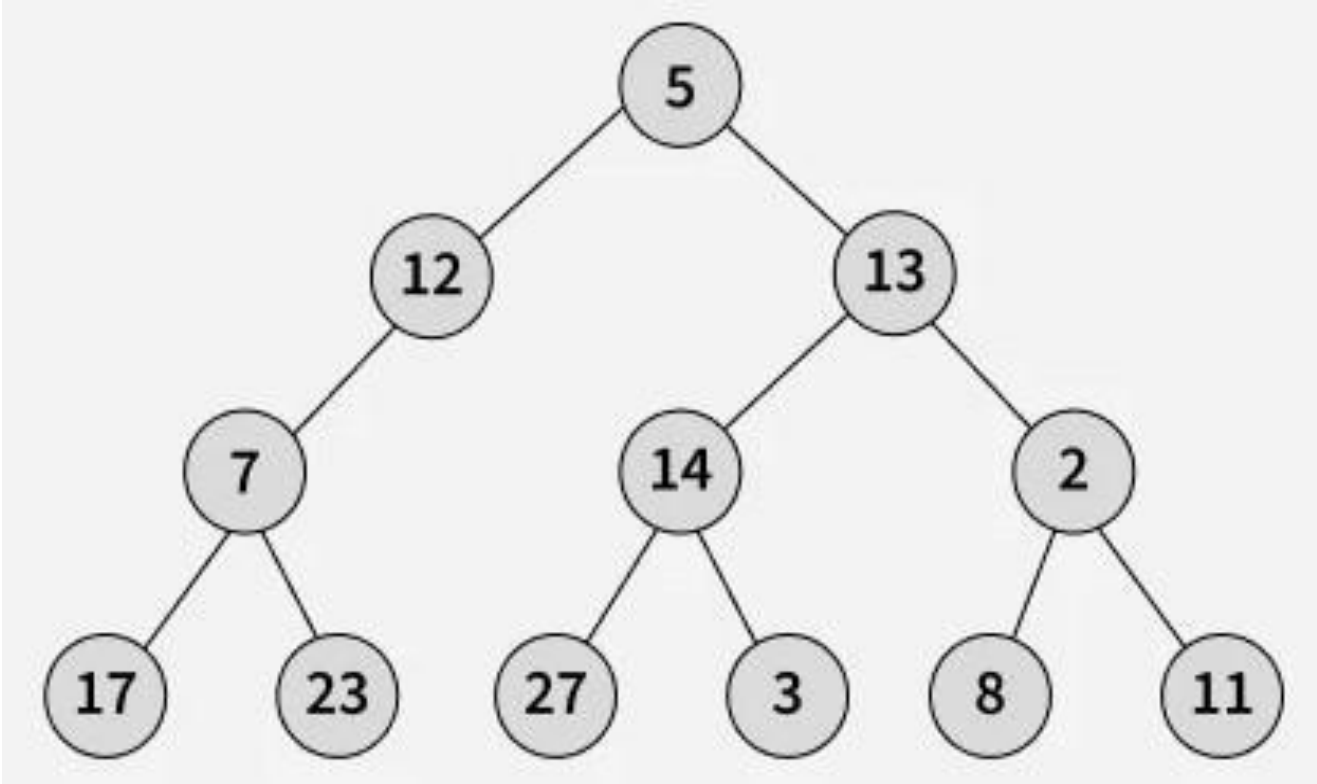


Breadth First Search

Given a **root** of the Binary Tree, find its **Level Order Traversal**.

Level Order Traversal technique is a method to traverse a Tree such that all nodes present in the same level are traversed completely before traversing the next level.

Example:



Output: *[[5], [12, 13], [7, 14, 2], [17, 23, 27, 3, 8, 11]]*
Explanation: *Start with the root - [5]*
Level 1: [12, 13]
Level 2: [7, 14, 2]
Level 3: [17, 23, 27, 3, 8, 11]

What is Breadth First Search?

Breadth First Search (BFS) is a graph traversal algorithm that starts traversing the graph from the root node and explores all the neighboring nodes at the present depth prior to moving on to the nodes at the next depth level.

How does BFS Tree Traversal work?

In the context of a tree, BFS traversal works similarly.

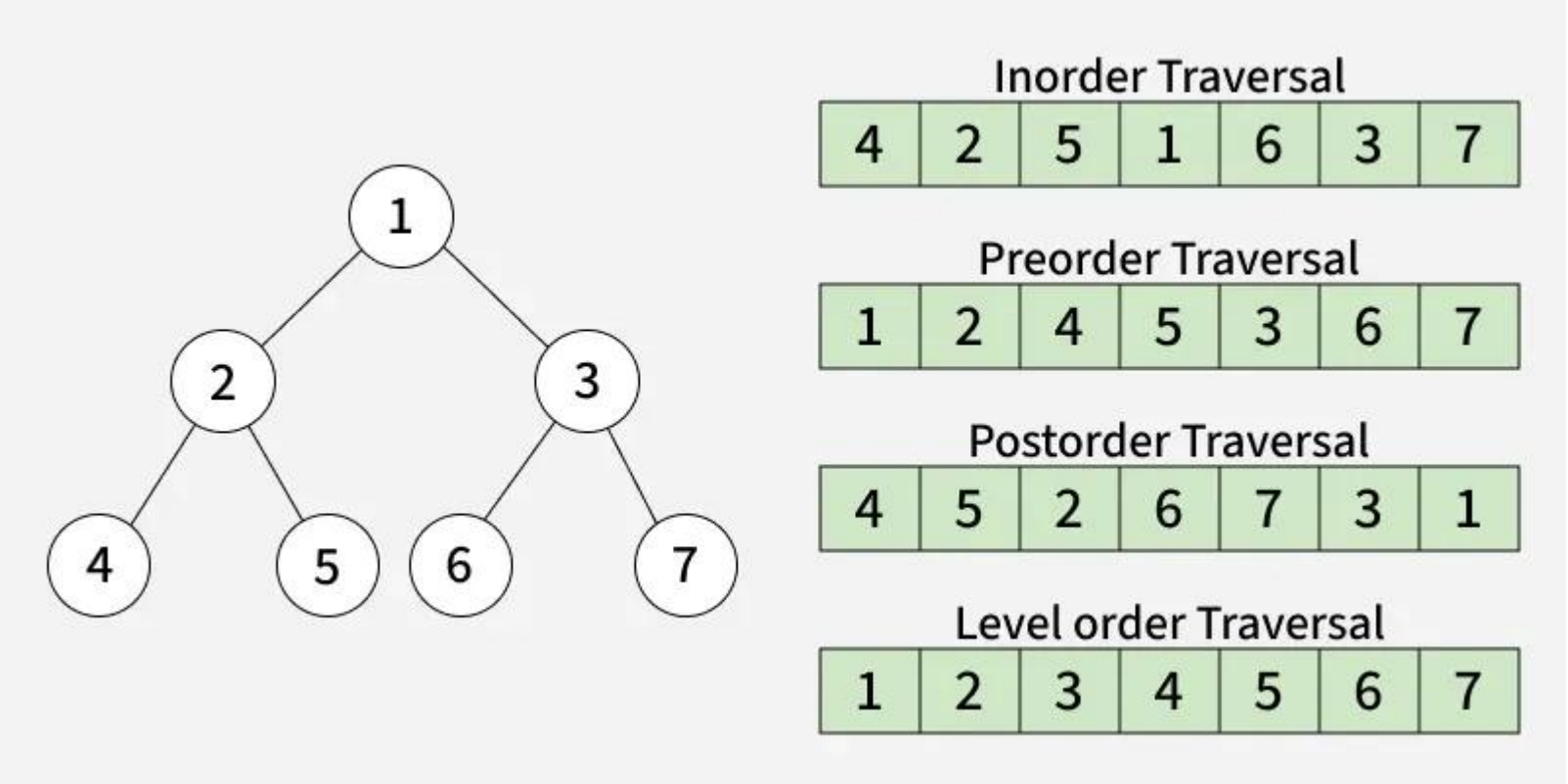
- Here's how BFS tree traversal typically works:
- Start at the root node and add it to a queue.
- While the **queue** is not empty, dequeue a node and visit it.
- Enqueue all of its children (if any) into the queue.
- Repeat steps 2 and 3 until the queue is empty.
- This approach ensures that nodes are visited level by level, moving horizontally across the tree before moving to the next level.

What is a Depth-first search?

DFS (Depth-first search) is a technique used for traversing trees or graphs. Here backtracking is used for traversal. In this traversal first, the deepest node is visited and then backtracks to its parent node if no sibling of that node exists.

Depth-First Traversal (DFT)

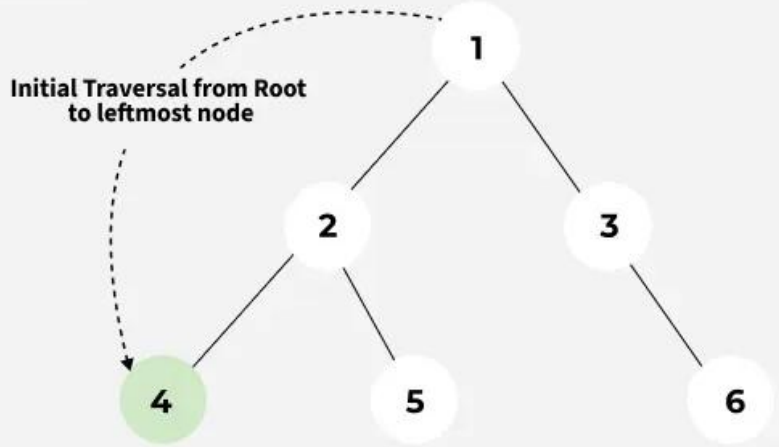
- Explores as far as possible along a branch before exploring the next branch.
- Types: Inorder, Preorder, Postorder



Inorder Traversal

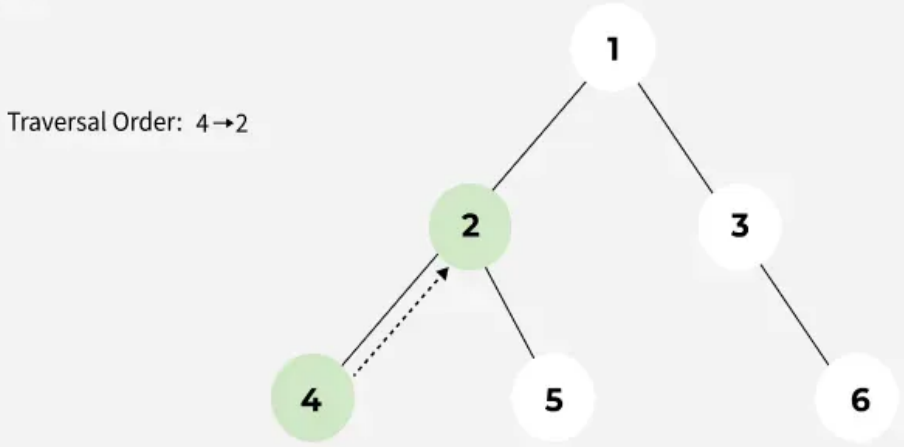
- Inorder traversal visits the node in the order: **Left -> Root -> Right**
- **Algorithm for Inorder Traversal**
- *Inorder(tree)*
  - *Traverse the left subtree, i.e., call Inorder(left->subtree)*
  - *Visit the root.*
  - *Traverse the right subtree, i.e., call Inorder(right->subtree)*

01 Step The traversal will go from 1 to its left subtree i.e., 2, then from 2 to its left subtree root, i.e., 4. Now 4 has no left subtree, so it will be visited. It also does not have any right subtree. So no more traversal from 4



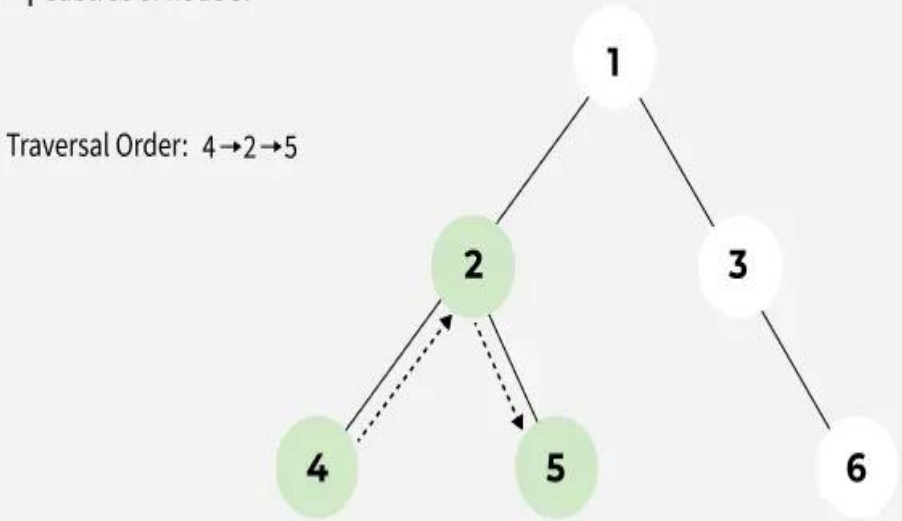
Inorder Traversal of Binary Tree

02 Step As the left subtree of 2 is visited completely, now it visits node 2 before moving to its right subtree.



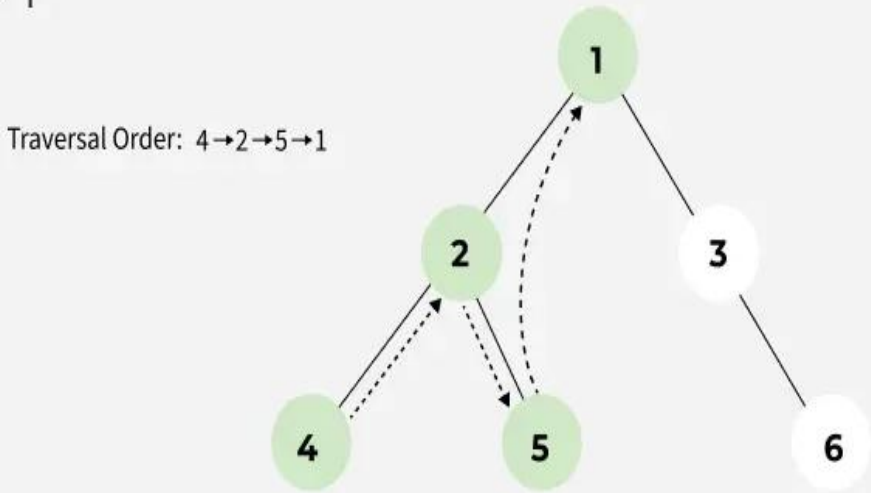
Inorder Traversal of Binary Tree

03 Step Now the right subtree of 2 will be traversed i.e., move to node 5. For node 5 there is no left subtree, so it gets visited and after that, the traversal comes back because there is no right subtree of node 5.



Inorder Traversal of Binary Tree

04 Step As the left subtree of node 1 is completely visited the root itself, i.e., node 1 will be visited.

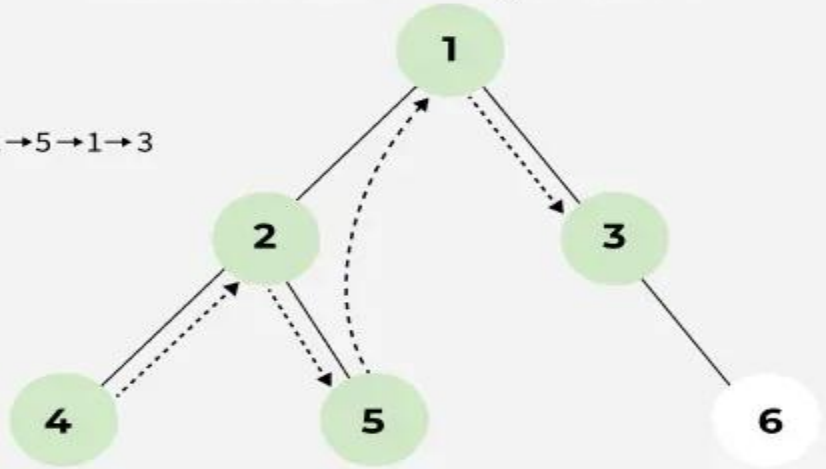


Inorder Traversal of Binary Tree

05 Step

Left subtree of node 1 and the node itself is visited. So now the right subtree of 1 will be traversed i.e., move to node 3. As node 3 has no left subtree so it gets visited.

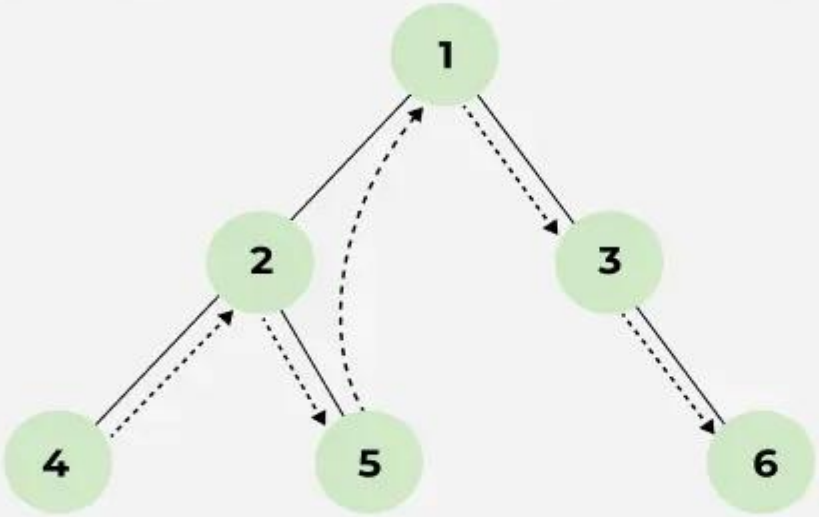
Traversal Order: 4→2→5→1→3



Inorder Traversal of Binary Tree

06 Step

The left subtree of node 3 and the node itself is visited. So traverse to the right subtree and visit node 6. Now the traversal ends as all the nodes are traversed.

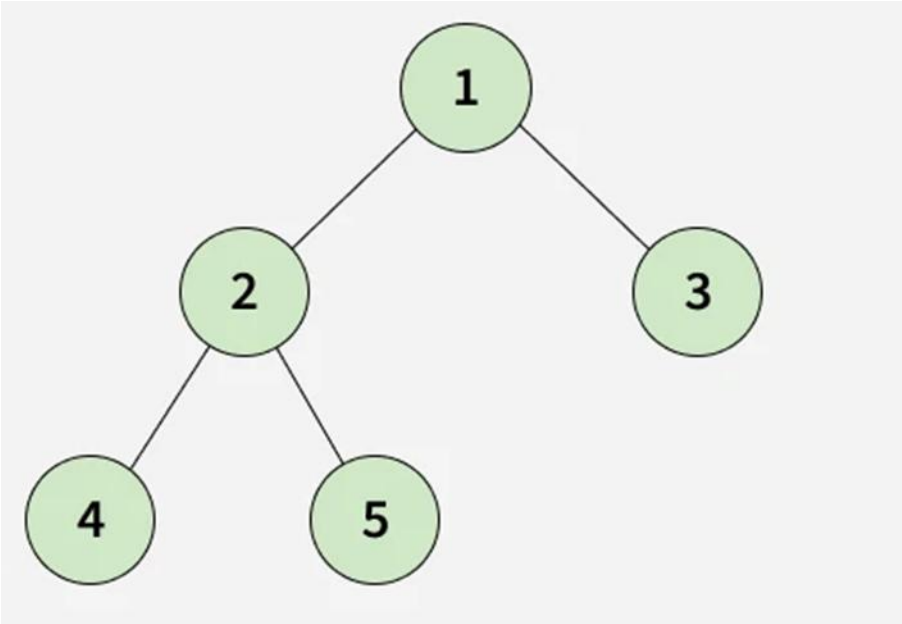


So the order of traversal of nodes is 4→2→5→1→3→6.

Inorder Traversal of Binary Tree

- Given a **root** of a Binary Tree, your task is to return its **Inorder Traversal**.
- **Note:** An **inorder traversal** first visits the left child (including its entire subtree), then visits the node, and finally visits the right child (including its entire subtree).
- **Examples:**

Input: root = [1, 2, 3, 4, 5]

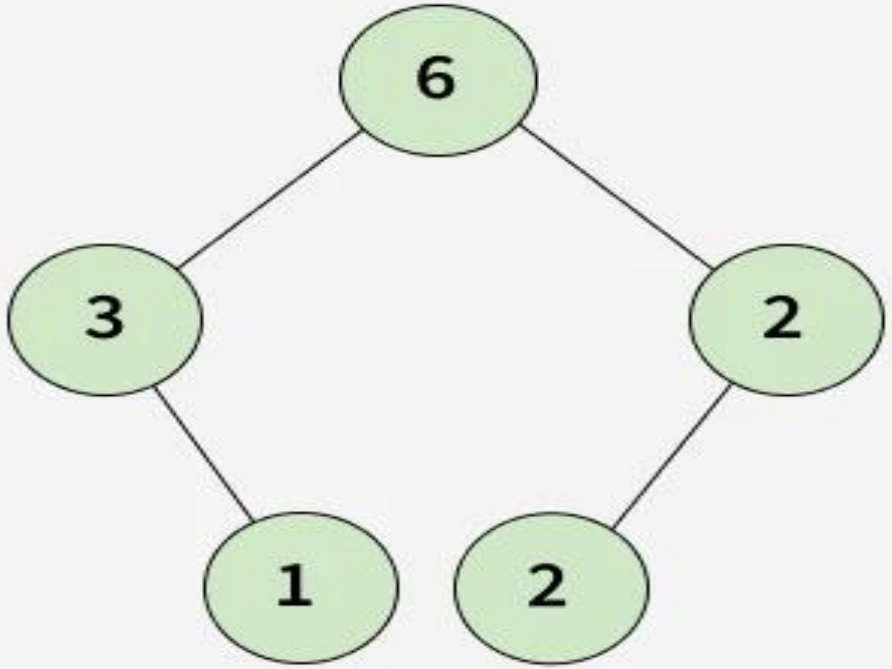


Output: [4, 2, 5, 1, 3]
Explanation: The inorder traversal of the given binary tree is [4, 2, 5, 1, 3].

2. Preorder Traversal:

- Follow the below steps to solve the problem:
- Visit the root
- Traverse the left subtree, i.e., call Preorder(left-subtree)
- Traverse the right subtree, i.e., call Preorder(right-subtree)
- Given the **root** of a binary tree, your task is to return its **Preorder traversal.**
- **Note:** A **preorder traversal** first visits the node, then visits the left child (including its entire subtree), and finally visits the right child (including its entire subtree).
- **Examples:**

Input: root = [6, 3, 2, N, 1, 2, N]



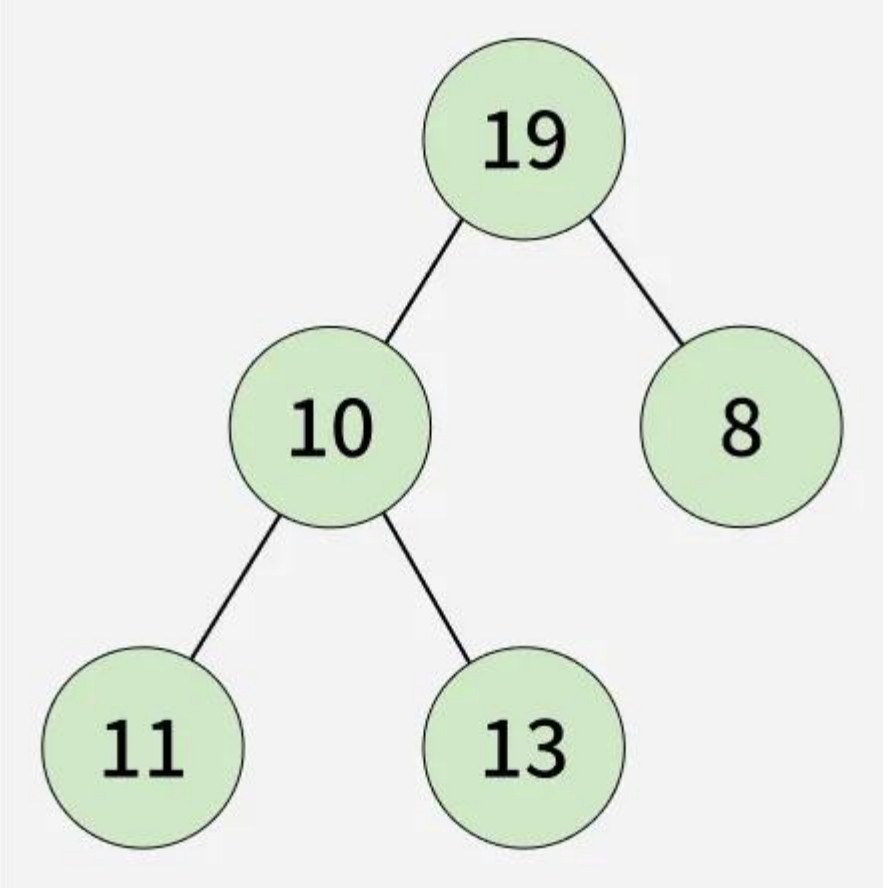
Output: [6, 3, 1, 2, 2]

Explanation: The preorder traversal of the given binary tree is [6, 3, 1, 2, 2]

3. Postorder Traversal:

- Follow the below steps to solve the problem:
- Traverse the left subtree, i.e., call Postorder(left-subtree)
- Traverse the right subtree, i.e., call Postorder(right-subtree)
- Visit the root.
- Given the **root** of a Binary Tree, your task is to return its **Postorder Traversal**.
- **Note:** A **postorder traversal** first visits the left child (including its entire subtree), then visits the right child (including its entire subtree), and finally visits the node itself.
- **Examples:**

Input: root = [19, 10, 8, 11, 13]



Output: [11, 13, 10, 8, 19]
Explanation: The postorder traversal of the given binary tree is [11, 13, 10, 8, 19].